Linear programming: models and certificates

Describe decisions with linear objectives and constraints. Distinguish optimality, infeasibility, and an unbounded objective.

Start with equivalent formulations and models, then use simplex, duality, and Phase I to build checkable certificates.

Learning route

Question	Method
How do we preserve the original solutions?	Convert to standard and slack forms.
How do we move while staying feasible?	Bases, pivots, and the ratio test.
How do we certify the result?	Dual certificates, infeasibility certificates, or improving rays.

Prerequisites include linear algebra, sets, and basic optimization concepts.

Standard and slack forms

Our standard form is $\max\ c^{\top}x,\ Ax \le b,\ x \ge 0$, with $A \in R^{(m \times n)}$. Constraints are non-strict and variables continuous real numbers. Finite binary-input bit complexity assumes rational coefficients. Other texts call equality form standard; check conventions first.

A feasible point satisfies all constraints; an optimum maximizes the objective over feasible points. Infeasible means no feasible point. Unbounded means feasible with arbitrarily large objective, not merely an unbounded feasible region. For example, $\max(-x)$ over $x \ge 0$ has an unbounded region and optimum 0.

Intersections of finitely many linear half-spaces are convex polyhedra:$Ax \le b$, $Ay \le b$ imply $A(\lambda x+(1-\lambda)y) \le b$ for $0 \le \lambda \le 1$. Convexity neither guarantees integer solutions nor makes nonlinear products linear constraints.

Four conversions must recover original variables

Convert minimizing f to maximizing−f and negate the returned objective. Multiply $a^Tx \geq b$ by−1. Split equality into opposite inequalities. Replace unrestricted u by $u^+ - u^-$ with both nonnegative. Every u has representation(max(u,0), max(−u,0)); every such pair recovers an original value by subtraction. Although not one-to-one, the optimum values correspond.

For `min 2u-v; u+v=3; u-2v≥-3; v≥0; u free`, maximize$-2u^+ + 2u^- + v$ subject to

$u^+ - u^- + v \leq 3;\ -u^+ + u^- - v \leq -3;\ -u^+ + u^- + 2v \leq 3$.

Substituting u=3−v gives $v \leq 2$ and original objective 6−3v, so v=2, u=1 attains minimum 0, corresponding to maximum 0. Starting with n variables, m constraints, sign splitting uses at most 2n variables and equality splitting at most 2m constraints, adding only constant coefficient bit length.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Slack form, bases, and basic feasibility

Introduce s=b−Ax≥0. Let M=[A I_m], q=(x, s)≥0, so Mq=b. Slacks are uniquely determined, not freely allocated independent decisions.

Choose m independent columns as basis B and let N be the others. Invertibility gives

$$q_B = M_B^{-1} b - M_B^{-1} M_N q_N, \qquad z = v + \rho_N^T q_N.$$

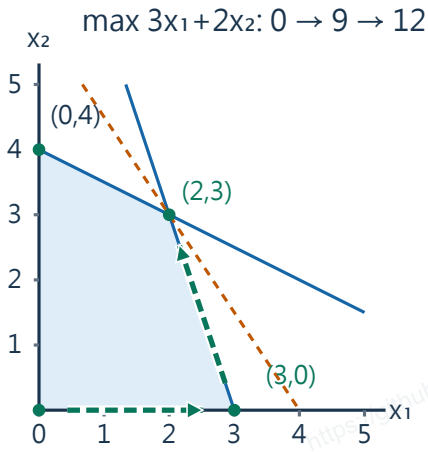
Set all nonbasic variables to 0 for a basic solution. It is feasible exactly when $M_B^{-1} b \geq 0$ componentwise. Initial slack basis I is feasible exactly when b≥0. Basic values need not be positive; zero basic variables cause degeneracy. A fixed basis determines a unique dictionary because the inverse is unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometry, level lines, and basis on one example

Use $\max\ 3x_1+2x_2;\ x_1+2x_2\le 8;\ 3x_1+x_2\le 9;\ x\ge 0$. Feasible vertices are(0,0),(3,0),(2,3),(0,4). The vertex path improves objective 0→9→12, ending at(2,3).

Initially $x_3=8-x_1-2x_2;\ x_4=9-3x_1-x_2;\ z=3x_1+2x_2$, with basis{$x_3$, x_4} and slacks(8,9).



blue: resource constraints

$x_1+2x_2=8$

$3x_1+x_2=9$

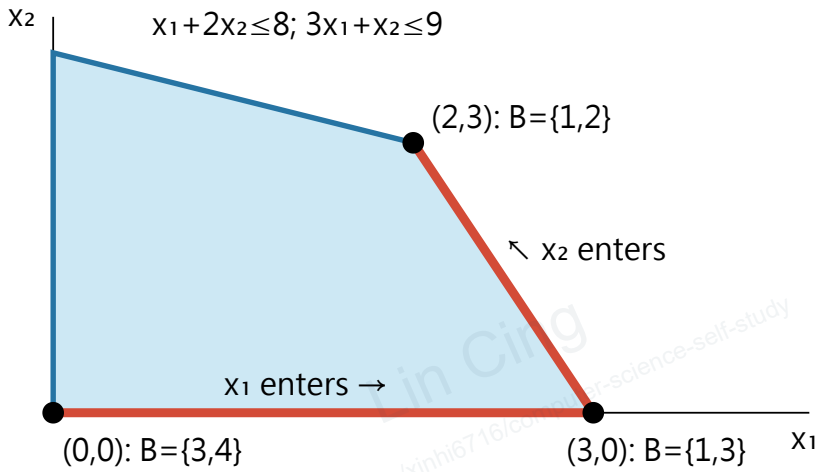
dashed arrows: two simplex moves

orange dashed: $3x_1+2x_2=12$

intersection (2,3), objective 12

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Geometry, level lines, and basis on one example (continued)



An entire edge can be optimal. For objective x_1+2x_2, the segment(0,4) to(2,3) has value 8. A vertex method returns one optimal basic solution without asserting uniqueness.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Formulating linear programs

Specify each variable's meaning, unit, and domain. Map every model-feasible solution to an original plan and every original feasible plan back. Matching objectives while omitting capacity or conservation does not give equivalent models.

For shortest paths, use potentials d_v, fix d_s=0, and constrain $d_v - d_u \leq w_{uv}$ per edge. Maximize d_t. Potentials are unrestricted; negative paths can require d_t<0.

First assume all vertices are source-reachable, no negative cycle exists, and t is reachable. Summing inequalities along any s-to-t path gives $d_t \leq$ its length. Every feasible potential is therefore a lower bound on the shortest distance. Actual distances δ(s, v) satisfy all triangle inequalities and attain δ(s, t). Maximization, not minimization, finds the shortest distance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential boundaries are not program Infinity values

Restricting to the s-reachable subgraph gives finite true distances when no negative cycle exists. Report unreachable t separately;∞ is not a real LP variable. Keeping an unreachable subgraph with a negative cycle instead makes the whole potential LP infeasible even if the source-to-target distance is ordinary. That assumption cannot be omitted.

If the entire graph has no negative cycle, a zero-edge super-source gives globally feasible potentials. For t unreachable from s, increase every unreachable vertex's potential by arbitrary K>0. No reachable-to-unreachable edge exists, so all differences stay feasible. Then d_t is unbounded, corresponding to unreachability, not a negative-length path.

For s→a:2, a→t:−1, s→t:4, constraints are $d_a \leq 2$, $d_t \leq d_a - 1$, $d_t \leq 4$. Values(0,2,1) are feasible and the two-edge path bounds $d_t \leq 1$, so optimum 1. Erroneously requiring all d≥0 would exclude correct solutions on other negative-total-path examples.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Flow models use nonnegative variables per original arc

Use arc flow g_{uv} rather than the preceding chapter's antisymmetric net flow. Constrain $0 \leq g_{uv} \leq c_{uv}$, internal outflow minus inflow 0, and maximize net source outflow. Antiparallel original arcs each retain their own variable and capacity. Do not also impose $g_{uv} = -g_{vu}$.

Every original flow satisfies the model, and capacity/conservation make every model solution a flow. There are m variables and O(n+m) constraints, without n^2 nonedge variables.

Minimum-cost flow additionally fixes source net outflow D and sink net inflow D, and minimizes $\sum_{(u,v)} a_{uv} g_{uv}$. Demand is exact; maximizing flow then reporting cost does not solve the same objective. Finite original capacities bound negative-cost circulation, which need not make the LP unbounded. Prohibiting circulation requires an additional model restriction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete minimum-cost-flow example

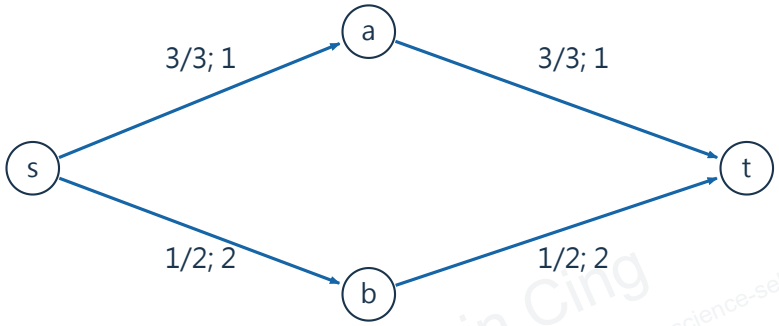
Arcs s→a, a→t each have capacity 3, unit cost 1. Arcs s→b, b→t each have capacity 2, unit cost 2. Demand D=4. Conservation makes route flows u, v, with u+v=4,$0 \leq u \leq 3$,$0 \leq v \leq 2$, and objective min 2u+4v.

Substitute v=4−u to get cost 16−2u and feasible $2 \leq u \leq 3$. Thus u=3, v=1 costs 10. This specifies every arc flow, demand, feasible interval, and optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete minimum-cost-flow example (continued)

Demand D=4; edge label: flow / capacity; unit cost



u+v=4

2≤u≤3

cost = 16−2u

optimal: u=3, v=1

upper: 3×(1+1)=6; lower: 1×(2+2)=4; total 10

Using u+v≤4 would incorrectly make zero flow at cost 0 optimal. Omitting u≤3 would incorrectly select u=4 at cost 8.

Multicommodity flow shares capacities

Commodity k has(s_k, t_k, D_k) and per-arc $g_{uv}^k \geq 0$. Each commodity separately conserves flow and meets endpoint net supplies$\pm D_k$. Shared capacity is $\sum_k g_{uv}^k \leq c_{uv}$. A feasibility objective may be0; costs can also be included.

Two commodities each uniquely requiring a capacity 1 bottleneck u→v with demand 3/4 are individually feasible but jointly need 3/2>1. Independent max-flow solutions miss this shared resource.

With k commodities, m arcs, n vertices, there are km variables and O(kn+m) constraints, a polynomial-size LP for rational data. This models splittable flow. One-path-per-commodity or integral-flow requirements create different discrete problems, without the same general LP guarantees.

Simplex dictionaries and the ratio test

Fix the sign convention below: a_{ij} is the coefficient **being subtracted** in a dictionary row.

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j, \qquad z = v + \sum_{j \in N} c_j x_j.$$

From a feasible dictionary with $b_i \geq 0$, choose $c_e > 0$ to enter, set $x_e = \theta \geq 0$, and keep the other nonbasic variables zero. Each basic value becomes $x_i = b_i - a_{ie}\theta$.

If $a_{ie} > 0$, feasibility requires $\theta \leq b_i / a_{ie}$; if $a_{ie} \leq 0$, the row imposes no upper bound. Choose the **minimum nonnegative ratio, including zero**, among positive denominators.

If there is no positive denominator, set $q_e = 1$, $q_i = -a_{ie} \geq 0$, and the other $q_j = 0$. The ray $x(\theta) = x(0) + \theta q$ is feasible and its objective $v + c_e\theta \to \infty$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pivot: solve for the entering variable

Let e enter and l leave, with $a_{le} \neq 0$. Solve row l exactly:

$$x_e = \frac{b_l}{a_{le}} - \frac{x_l}{a_{le}} - \sum_{j \in N \setminus \{e\}} \frac{a_{lj}}{a_{le}} x_j.$$

Substitute this row into the other rows and the objective, then exchange the basic set B and nonbasic set N. Variable x_l becomes nonbasic but remains in the dictionary. Omitting $-x_l / a_{le}$ changes the feasible set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pivot: update the coefficients

For the new entering row,

$$a'_{el} = \frac{1}{a_{le}}, \quad b'_e = \frac{b_l}{a_{le}}, \quad a'_{ej} = \frac{a_{lj}}{a_{le}}.$$

For the remaining $i \neq l$ and $j \in N \setminus \{e\}$,

$$b'_i = b_i - a_{ie} b'_e, \quad a'_{il} = -\frac{a_{ie}}{a_{le}}, \quad a'_{ij} = a_{ij} - \frac{a_{ie} a_{lj}}{a_{le}}.$$

The objective row gives

$$v' = v + c_e b'_e, \quad c'_l = -\frac{c_e}{a_{le}}, \quad c'_j = c_j - \frac{c_e a_{lj}}{a_{le}}.$$

This invertible substitution preserves all points satisfying the equations and their objective values. Adding nonnegativity yields the same LP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the ratio test preserves feasibility

The new entering value is $b'_e = b_l / a_{le} = \theta \geq 0$. The other basic values are

$$b'_i = b_i - a_{ie} \theta.$$

For $a_{ie} > 0$, the minimum ratio ensures $\theta \leq b_i / a_{ie}$, so $b'_i \geq 0$. For $a_{ie} \leq 0$, we have $b'_i \geq b_i \geq 0$. The new nonbasic variables are zero, so the new basic solution is feasible.

The objective changes by $c_e \theta \geq 0$. It strictly improves if $\theta > 0$; at $\theta = 0$, the point and objective may remain unchanged while the basis changes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Simplex control flow

```
while some reduced cost c[e] > 0:
    choose entering e
    candidates = {i in B : a[i,e] > 0}
    if candidates empty: return feasible point + improving ray
    l = minimum-ratio row, using an anti-cycling rule
    pivot(l,e)
return current basic solution
```

A zero ratio must remain eligible. If there is no leaving candidate, the improving ray certifies unboundedness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two pivots on the same two-dimensional example

Initially B={3,4}. Enter x_1:ratios 8/1=8,9/3=3 make x_4 leave with $\theta=3$:

$x_1=3-x_2/3-x_4/3;\ x_3=5-5x_2/3+x_4/3;\ z=9+x_2-x_4.$

Enter x_2:ratios 3/(1/3)=9,5/(5/3)=3 make x_3 leave:

$x_2=3-3x_3/5+x_4/5;\ x_1=2+x_3/5-2x_4/5;\ z=12-3x_3/5-4x_4/5.$

Nonbasic $x_3=x_4=0$ gives(2,3), tight in both constraints. Every reduced cost is nonpositive, so every feasible point has objective≤ 12. No further numerical search of the polygon is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zero steps are legal pivots

Use $\max\ z=x_1;\ x_3=-x_1;\ x_4=1-x_1$, all variables nonnegative. Initially B={3,4}, N={1}, basic values(0,1).

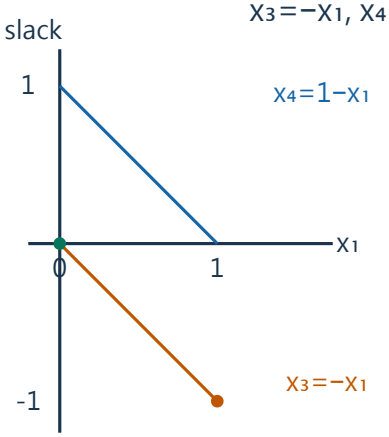
Basic row	Positive denominator	Ratio	At θ=0	Incorrect θ=1
x_3	1	0	0	−1, infeasible
x_4	1	1	1	0

Let x_3 leave. New $x_1=-x_3$, $x_4=1+x_3$, $z=-x_3$ has the same point(0,0,1), objective 0, but now nonpositive reduced costs prove optimality. Ignoring zero ratios immediately violates feasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zero steps are legal pivots (continued)

$x_3=-x_1$, $x_4=1-x_1$: do not skip a zero ratio



$x_1=0$: $x_3=0$, $x_4=1$, feasible

ratios: 0/1=0 and 1/1=1

feasible step θ=0; x_3 leaves

wrong step θ=1: $x_3=-1$, infeasible

only $x_1=0$ is feasible

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finitely many bases alone do not prevent cycling

Matrix M=[A I] has m rows, and a basis selects m independent columns. There are at most C(n+m, m) possibilities. A fixed basis determines one dictionary, so an infinite run repeats a basis. Degenerate steps may preserve objective, so mere nondecrease does not forbid repetition.

Use exact lexicographic perturbation for a self-contained anti-cycling proof. From an initial feasible dictionary $q_B = \beta - Dq_N$, replace its RHS by $\beta_i+\varepsilon^i$ for i=1,…, m. Here ε is a positive infinitesimal,**not an arbitrary floating decimal**. Nonnegative constants and a positive first ε coefficient make the initial dictionary strictly feasible in the ordered field.

Represent comparisons by coefficient vectors:constant first, then ε,ε^2,…; the first nonzero coefficient determines sign. Continue minimum-ratio tests with positive denominators, using later coefficients to resolve equal real ratios.

Bland’ s rule fixes variable indices: choose the smallest index with positive reduced cost, and among minimum nonnegative-ratio ties choose the smallest-index leaving variable. The finite-termination theorem assumes exact arithmetic and both choices; tie-breaking alone is insufficient.

Why the lexicographic rule terminates

Let the initial equation matrix be M_0=[I D], with RHS $\beta + \sum_i e_i\epsilon^i$. For any invertible basis K, its basic values are

$$M_{0,K}^{-1}\beta + \sum_{i=1}^{m}(M_{0,K}^{-1}e_i)\epsilon^i.$$

No inverse row is entirely zero, so no basic value is the identically-zero polynomial. Feasibility therefore makes every basic value strictly positive. Two positive-denominator ratios cannot coincide identically, or two inverse rows would be proportional, contradicting invertibility. Every legal θ is strictly positive, and c_e>0 makes the lexicographic objective strictly increase.

A fixed basis has one dictionary/objective, so it cannot repeat. Finite bases imply termination. Taking constant coefficients returns an original-LP feasible basic point. Reduced costs do not depend on RHS; nonpositive costs establish its optimality. An improving ray likewise has ε-independent direction and certifies original unboundedness from the constant feasible point. No guessed sufficiently-small floating ε is required.

A reproducible lexicographic tie

Use $x_3=2-x_1;\ x_4=2-x_1-x_2;\ \max\ x_1+x_2$, with RHS $2+\varepsilon, 2+\varepsilon^2$. Entering x_1 compares $2+\varepsilon$ against $2+\varepsilon^2$; the latter is smaller, so x_4 leaves.

New rows are $x_1=2+\varepsilon^2-x_2-x_4;\ x_3=\varepsilon-\varepsilon^2+x_2+x_4;\ z=2+\varepsilon^2-x_4$. No reduced cost is positive. The constant solution is $x_1=2$, other variables 0, objective 2. Although x_3 has zero constant, $\varepsilon-\varepsilon^2$ is strictly positive and separates degeneracy symbolically.

Another common anti-cycling choice is Bland's rule:smallest-index positive reduced-cost entering variable, and smallest-index leaving variable among minimum-ratio ties. Both parts matter. The complete termination proof here uses lexicographic perturbation and does not claim arbitrary choices terminate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Per-pivot cost is not total simplex cost

A dense ordinary dictionary updates O(mn) coefficients per pivot. This does not prove polynomial total time; basis count C(n+m, m) can be exponential. Lexicographic RHS adds m+1 coefficients per row and a polynomial update/comparison factor, but retains only a finite/exponential general worst-case pivot guarantee.

Rational exact pivots use fractions. Basis solutions are ratios of subdeterminants. After clearing original denominators, a determinant has at most m! terms, giving bit length bounded polynomially by dimension and input bit length. A single basis thus needs no unrestricted representation growth, but pivot count need not be polynomial.

Floating implementations need separate feasibility/reduced-cost tolerances, near-zero ratio handling, and residual recomputation. Clamping negative b to 0 can change the problem. Ellipsoid and interior-point methods are different algorithms; their polynomial theory does not follow from these pivot counts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duality

Primal `max cᵀx, Ax≤b, x≥0` has dual `min bᵀy, Aᵀy≥c, y≥0`. Its m resource rows give m prices and n primal variables give n price inequalities. Transpose A; merely exchanging max/min is insufficient.

For feasible x, y, nonnegativity and both constraint systems give

$$c^T x \leq x^T A^T y = y^T A x \leq y^T b.$$

This is weak duality. A dual feasible point bounds every primal objective above. Equality certifies optimality on both sides. Equal objective numbers without checking feasibility are not a certificate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Constructing a dual from the terminal basis

Let M=[A I], d=(c,0), and terminal basis matrix M_B be invertible. Define row $y^T = d_B^T M_B^{-1}$. Substituting basic variables gives

$$z = y^T b + \sum_{j \in N} (d_j - y^T M_j) q_j.$$

For every column, reduced cost $\rho_j = d_j - y^T M_j$, and basic costs are 0. Terminal $\rho \leq 0$ gives $-y_i \leq 0$ from slack columns e_i, hence $y \geq 0$. Decision columns give $c_j - y^T A_j \leq 0$, hence $A^T y \geq c$.

The current feasible basic objective is $y^T b$, attaining weak-duality equality. A finite optimal primal therefore has a same-value dual. Read y_i as the negative reduced cost of original slack i, or0 if that slack is basic. This follows from the full dictionary identity, not coefficient matching at only one point.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same example's dual and complementary slackness

The two-dimensional example has dual `min 8y₁+9y₂; y₁+3y₂≥3; 2y₁+y₂≥2; y≥0`. Terminal `z=12−3x₃/5−4x₄/5` gives y=(3/5,4/5). Both dual rows are tight, and 24/5+36/5=12 matches x=(2,3).

The general gap decomposes exactly as

$$b^T y - c^T x = y^T(b - Ax) + x^T(A^T y - c).$$

For feasible sides, every summand is nonnegative. Gap 0 is therefore equivalent to all products $y_i(b_i - A_i x) = 0$ and $x_j((A^T y)_j - c_j) = 0$. This proves necessity and sufficiency of complementary slackness. Positive primal slack forces its price 0; positive primal variable forces its dual row tight.

For `max 2x₁+x₂; x₁+x₂≤4; x₁≤2; x≥0`, dual y=(1,1) gives upper bound 6. Point(2,2) has value 6 and all complementary products 0. Point(1,3) has value 5, with the sole positive product $y_2(2-x_1)=1$.

The same example's dual and complementary slackness (continued)

$$\text{gap} = y^T(b-Ax) + x^T(A^Ty-c) \geq 0$$

x=(2,2)

y=(1,1), dual value 6

primal 6

$y_2(2-x_1) = 0$

other three products are 0

x′=(1,3)

y=(1,1), dual value 6

primal 5

$y_2(2-x_1) = 1$

other three products are 0

Unboundedness and infeasibility are not interchangeable

An unbounded primal has no feasible dual, or one fixed dual value would bound all primal values. A downward-unbounded dual similarly makes primal infeasible. The converse need not hold:an infeasible primal need not have an unbounded dual, because both can be infeasible.

Primal `max -x; 0x≤-1; x≥0` is infeasible; dual `min -y; 0y≥-1; y≥0` is feasible and downward-unbounded. Changing primal objective to max x gives dual constraint $0 \geq 1$, making both sides infeasible.

For other sign conventions, convert to standard first. A primal equality has a free dual multiplier; an unrestricted primal variable gives a dual equality. These rules follow by combining split nonnegative variables or opposite inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The initial basic feasible solution

Negative entries in initial b prevent starting a ratio loop that assumes feasibility. For $Ax \leq b$, $x \geq 0$, add $a \geq 0$ and solve auxiliary `max -a; Ax-a1≤b`. It is always feasible:take x=0, a=max(0,$-\min_i b_i$). Its objective is at most 0 and cannot be upward-unbounded.

Original feasibility allows the same x with a=0, attaining auxiliary optimum 0. Conversely, auxiliary optimum 0 gives a=0 and an original feasible x. A strictly negative optimum proves infeasibility. A currently positive artificial variable alone is insufficient; first establish auxiliary optimality.

When m=0, use the origin directly. A positive c_j gives an unbounded coordinate ray; otherwise the origin is optimal. No minimum over an empty row set is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A special pivot initializes auxiliary feasibility

Write s_i=b_i−$A_i x$+a and choose r with minimum b_r<0. Let a enter, s_r leave. The pivot coefficient is−1; this is **not** the improving ratio step from an already-feasible basis.

Solving gives $a = -b_r + A_r x + s_r$; other rows become $s_i = b_i - b_r + (A_r - A_i)x + s_r$. Set new nonbasic x=$s_r$=0. Then a=−$b_r$>0 and s_i=b_i−b_r≥0, establishing a feasible basis. Only now use simplex and lexicographic anti-cycling.

Start symbolic RHS perturbation at this feasible dictionary and take constant parts afterward. Reduced costs ignore RHS, so the result is the original auxiliary optimum and feasible point, rather than a false infeasibility judgment from an ε objective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zero artificial value still requires basis cleanup

At auxiliary optimum 0, delete a's column if it is nonbasic. If a is basic with value 0, its row is $a = -\sum_j a_{aj} x_j$. A nonzero coefficient permits that x_j to enter and a to leave. This zero-step algebraic pivot leaves the point unchanged; its coefficient may have either sign and it is not an improving ratio step.

If every coefficient is 0, the equation a=0 is redundant. Delete its row and a. General equality or multi-artificial implementations need this branch. Full independent slack rows can sometimes exclude it by rank, but never justify division by 0.

Finally substitute every currently basic variable appearing in the **original objective** to recover a nonbasic-only objective. Changing a title or retaining auxiliary−a does not restore the original problem. This completes Phase I; Phase II begins with that feasible basis and restored objective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete Phase I and original-objective example

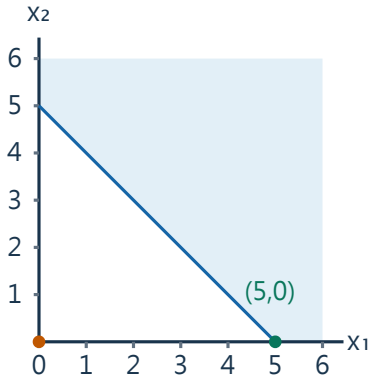
Original $\min\ z=2x_1+x_2;\ x_1+x_2\geq 5;\ x\geq 0$ uses surplus $x_3\geq 0$ with $x_1+x_2-x_3=5$. Add artificial $a=5-x_1-x_2+x_3$, minimizing $w=a$ in Phase I, equivalent to maximizing $-a$.

State	Basic row	At nonbasics 0	Objective
Initial	$a=5-x_1-x_2+x_3$	$a=5$	$w=5$
x_1 enters, a leaves	$x_1=5-x_2+x_3-a$	$x_1=5$	$w=a$, min 0
Delete a; substitute original objective	$x_1=5-x_2+x_3$	$x_1=5$	$z=10-x_2+2x_3$
x_2 enters, x_1 leaves	$x_2=5-x_1+x_3$	$x_2=5$	$z=5+x_1+x_3$

Nonnegative final minimization coefficients prove optimum 5. Original origin(0,0) is infeasible; adding a=5 makes it feasible only for the auxiliary problem. This one-equation example uses one artificial column; the shared-a construction above covers arbitrary $Ax\leq b$ systems.

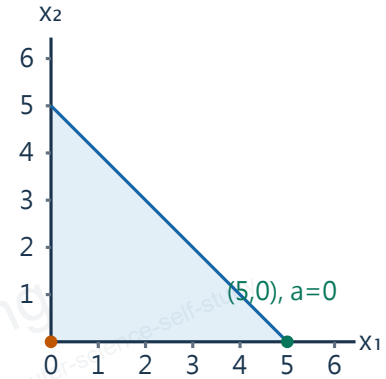
Complete Phase I and original-objective example (continued)

Original: $x_1+x_2\geq 5$



origin is infeasible

Auxiliary slice $x_3=0$: $a=5-x_1-x_2$



origin feasible with a=5

The auxiliary problem returns to the original constraint only when a=0

Infeasibility needs a completed certificate

Original $x\le 1$, $x\ge 2$, $x\ge 0$ becomes $x-a\le 1$, $-x-a\le -2$. Summing gives $-2a\le -1$, so $a\ge 1/2$. Values $x=3/2$, $a=1/2$ satisfy both tightly, proving auxiliary max $-a=-1/2<0$ and original infeasibility.

The complete dictionary starts $s_1=1-x+a$, $s_2=-2+x+a$. Special a-entry, s_2-exit gives $a=2-x+s_2$, $s_1=3-2x+s_2$, $w=-2+x-s_2$. Enter x:ratios 2,3/2 make s_1 leave, producing

$x=3/2-s_1/2+s_2/2;\ a=1/2+s_1/2+s_2/2;\ w=-1/2-s_1/2-s_2/2$.

Both reduced costs are nonpositive, so this is truly optimal. A positive artificial amount alone would not suffice. The original inequalities also sum to $0\le -1$, an independent contradiction certificate. More generally, $y\ge 0$, $A^Ty\ge 0$ and $b^Ty<0$ certify infeasibility: any feasible x would imply $0\le x^TA^Ty=y^TAx\le y^Tb<0$. Here $y=(1/2,1/2)$, $A^Ty=0$ and $b^Ty=-1/2$.

Three outcomes and the proof dependencies

Phase I starts from a feasible auxiliary basis. Lexicographic simplex terminates finitely, and auxiliary objective is bounded above. A negative optimum means infeasible; a zero optimum permits artificial cleanup and objective restoration.

Phase II also starts feasible and terminates under the lexicographic rule. No positive ratio denominator produces a feasible point plus improving ray, certifying unboundedness. Otherwise nonpositive terminal reduced costs produce a feasible basic point and same-value dual, certifying a finite attained optimum.

Thus every LP in this standard form is exactly one of infeasible, feasible with unbounded objective, or feasible with a finite attained optimum. Finite-dimensional structure and non-strict constraints matter. Problem $\max\ x;\ 0\le x<1$ has unattained supremum 1 and lies outside this trichotomy's assumptions.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, Chapter 29. MIT 6.046J Fall 2004, Lecture 15; Fall 2005, Lecture 18.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study